HATHOR-CORE · MASTER-BOOK

Observability — Metrics, Profiler & Health

The three operator-facing windows into a running node — Prometheus metrics, the CPU profiler, and the health-check endpoint that tells orchestrators it is alive.

SUBJECT	hathor-core · Part II · the node, end to end
CHAPTER	42 · Part II · The Node
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

Observability · Prometheus · Metrics · Counter/Gauge · Pull scraping · CPU profiler · Reactor stalls · Health check · Liveness/readiness · Kubernetes

Table of Contents

Chapter 42 — Observability: Metrics, Profiler & Health 3

42.1 Localization 3

42.2 What it does and why it exists 4

The four questions 5

42.3 The concepts it rests on 5

42.4 The code, walked — Metrics and Prometheus 6

42.4.1 What a metric is 6

42.4.2 Pull-based scraping — the Prometheus model 7

42.4.3 Two objects: Metrics and PrometheusMetricsExporter 8

42.4.4 The twist: file-based export, not an HTTP endpoint 9

42.4.5 Why Prometheus, and why pull 10

42.5 The code, walked — the CPU profiler 11

42.5.1 The reactor-stall problem 11

42.5.2 SimpleCPUProfiler — a sampling marker stack 11

42.5.3 Where the marks come from — the @profiler decorator and SiteProfiler 12

42.5.4 The /top API and hathor-cli top 13

42.6 The code, walked — the health-check endpoint 13

42.6.1 Liveness versus readiness 13

42.6.2 What "healthy" means for a Hathor node 14

42.6.3 The two resources 14

42.7 How it plugs into the lifecycle 15

Recap 16

Chapter 42 — Observability: Metrics, Profiler & Health

What you'll learn in this chapter

- What **observability** is, and why a node you cannot watch is a node you cannot operate.
- The four questions an operator asks — what happened, how much, why is it slow, is it alive — and which tool answers each.
- **Prometheus and metrics**: what a counter, a gauge, and a histogram are; what pull-based scraping means; and exactly which numbers `hathor-core` exposes.
- The **CPU profiler**: why a single-threaded reactor makes a slow callback catastrophic, and how the profiler finds the offending code.
- The **health-check** endpoint: the difference between liveness and readiness, what makes a Hathor node "healthy," and how an orchestrator like Kubernetes uses it.
- How all three are wired into the node's lifecycle.

This is the second-to-last chapter of Part II. The node we have assembled across the preceding chapters now runs: it boots, syncs, verifies, reaches consensus, and serves clients. This chapter is about the windows cut into its side — the surfaces an operator uses to see what the running node is doing, so they can keep it healthy. The three packages are small. The concepts behind them are the part worth your time.

42.1 Localization

Three pieces, all in the infrastructure tier of the module map (Chapter 0, §0.4). Two are single files; one is a small package.

```
hathor/
├── metrics.py                    ← the Metrics object: samples node state   ◀ YOU ARE HERE
├── prometheus.py                 ← PrometheusMetricsExporter: writes the .prom file
│
├── profiler/
│   ├── __init__.py               ← get_cpu_profiler() singleton accessor
│   ├── cpu.py                    ← SimpleCPUProfiler: the measuring engine
│   ├── site.py                   ← SiteProfiler: profiles HTTP requests
│   └── resources/
│       └── cpu_profiler.py       ← CPUProfilerResource: the /top REST API
│
└── healthcheck/
    └── resources/
        └── healthcheck.py        ← HealthcheckResource: the /health endpoint
        (readiness lives at p2p/resources/healthcheck.py — /p2p/readiness)

hathor_cli/
└── top.py                        ← `hathor-cli top`: the curses viewer for the profiler
```

> **Context.** None of these packages changes the ledger. They do not verify, store, or sync a single vertex. They exist so that a human (or a machine acting for a human) can answer questions about the node from the outside. A mainnet node is a long-running server that must stay up for months; you cannot babysit it by hand. Observability is the set of surfaces that let you, or your monitoring system, watch the node's pulse, find out why it is slow, and decide whether to restart it — without ever stopping it to look inside.

42.2 What it does and why it exists

There is an old operations saying: **you cannot fix what you cannot see.** A program that runs correctly in front of you on your laptop is one thing. A program that runs for ninety days unattended on a server in another country, processing millions of transactions, is another. When that second program misbehaves — gets slow, stops keeping up with the network, leaks memory — you have no debugger attached, no `print` statements you can add without redeploying, and no way to stop it and look without taking the service down. Everything you learn about it, you must learn from surfaces it deliberately exposes while it keeps running.

That is what **observability** means: the property of a system that lets you understand its internal state purely from its external outputs. A car's dashboard is the everyday example. You do not open the engine to know your speed or your fuel level; the car exposes those numbers on a panel built for exactly that purpose. The metrics, profiler, and health-check surfaces are `hathor-core`'s dashboard.

The four questions

It helps to be precise about what you want to know, because different questions need different tools, and `hathor-core` ships a different surface for each. An operator staring at a running node asks four distinct questions:

Question	Tool	The chapter
What happened?	structured logs	Ch 17
How much, how fast?	metrics (Prometheus)	this chapter, §42.4
Why is it slow?	the CPU profiler	this chapter, §42.5
Is it alive / ready?	the health-check endpoint	this chapter, §42.6

The first question — what happened — is answered by **logs**, which we gave their full treatment in Chapter 17. A log is a record of discrete events: "block accepted," "peer disconnected," "verification failed." Logs are narrative; they tell you the story of what the node did, event by event. They are excellent for debugging a specific incident after the fact ("show me everything that happened around 14:32").

But logs are bad at the other three questions. If you want to know how many transactions the node is processing per second, reading a log line per transaction is hopeless — you would be counting by hand. If you want to know why the node is slow, a log tells you what it did, not where the time went. And if you want a yes/no answer to is this node healthy, you do not want to read a story at all; you want a single bit.

So observability is not one pillar but several. Logs are one (Chapter 17). This chapter covers the other three. The recurring slogan worth memorizing:

> **Logs answer WHAT. Metrics answer HOW MUCH.** A log entry is one event, written once, read by a human reconstructing a timeline. A metric is one number that changes over time, sampled repeatedly, read by a machine drawing a graph. You do not log "transaction count = 4,182,991" on every transaction; you expose it as a metric and let a monitoring system chart it. The two are complementary, never substitutes. → logs are Chapter 17; metrics are §42.4 below.

42.3 The concepts it rests on

Three ideas from earlier chapters underpin this one. We recap them in their local form here; the full treatments are where the pointers say.

> **Recap — structured logging (full treatment in Ch. 17).** `hathor-core` does not write log lines as freeform English. It writes them as key-value events: `event="new block" height=42 hash=00ab...`. This makes logs machine-queryable. Logs are the first observability pillar and answer "what happened"; this chapter covers the rest. → Chapter 17.

> **Recap — the single-threaded reactor (full treatment in Ch. 2 & 16).** The entire node runs its logic on one thread, driven by the Twisted **reactor**[1] — a single event loop that waits for an event (data on a socket, a timer firing), calls the one piece of your code that handles it, and only when that code returns moves on to the next event. The cardinal rule (Chapter 2) is never block the reactor: while one callback runs, every other event — every other peer, every API request, every timer — waits. A callback that takes two seconds freezes the whole node for two seconds. This single fact is why a CPU profiler matters so much here (§42.5). → Chapters 2 and 16.

> **Recap — the manager owns observability (full treatment in Ch. 29).** The `HathorManager` is the node's central coordinator. It constructs the `Metrics` object during its own `__init__` (`manager.py:212`) and calls `self.metrics.start()` inside its `start()` sequence (`manager.py:325`), stopping it on shutdown (`manager.py:360`). The metrics object is owned by the manager; the Prometheus exporter and the REST resources are wired separately by the builders (§42.7). → Chapter 29.

A note on the **`LoopingCall`**[2], which appears in all three surfaces. It is Twisted's periodic timer: "call this function every N seconds." Metrics, the Prometheus exporter, and the profiler all use one to wake up on a schedule and sample. Because it runs on the reactor, the sampling itself must be cheap — a long sample would block the node, the very thing we are trying to watch.

42.4 The code, walked — Metrics and Prometheus

This is the canonical treatment of metrics and Prometheus for the whole book. We build the concept generically first, then show Hathor's exact spelling.

42.4.1 What a metric is

A **metric** is a single named number that describes some aspect of a running system, and that you sample over time. "Number of connected peers." "Total transactions processed." "Bytes received from peer X." Each is one metric. Plot any of them against time and you get a graph; a wall of such graphs is a monitoring dashboard.

The monitoring world classifies metrics into three shapes, and the distinction is not pedantry — it tells the monitoring system how to interpret the number.

- A **counter** only ever goes up (until the process restarts, when it resets to zero). "Total transactions received since boot" is a counter. You never decrement it. What you actually chart is its rate of change — the slope — which gives you "transactions per second." Hathor's per-peer `received_messages`, `received_bytes`, and `received_txs` (`metrics.py:48–55`) are counters in spirit: monotonically rising tallies.
- A **gauge** can go up or down. It is a snapshot of a current level. "Number of connected peers right now" is a gauge — it rises when a peer connects, falls when one drops. "Best block height," "mempool size," "RocksDB column-family size in bytes": all gauges. Most of what `hathor-core` exposes is a gauge.
- A **histogram** buckets many observations of a value to show its distribution — "how long did requests take, grouped into <10ms, <100ms, <1s buckets." It answers "what does the spread look like," not just "what is the latest value." `hathor-core` does not currently expose histograms through its Prometheus surface; it leans entirely on counters and gauges. We define the shape here so the vocabulary is complete and so you recognize it in other systems.

Generic intuition before the real code — picture the simplest possible metric registry:

```
# A toy metric: a gauge you read whenever someone asks.
class Gauge:
    def __init__(self, name):
        self.name = name
        self.value = 0.0

    def set(self, v):       # a gauge can be set to any value
        self.value = v

peers = Gauge("connected_peers")
peers.set(8)                # 8 peers connected right now
# ... later, a monitoring system reads peers.value and charts it.
```

The real Prometheus client library gives you exactly this `Gauge` (plus `Counter`, `Histogram`), with the machinery to expose every gauge's current value in a format a monitoring server can read. Hathor uses that library.

42.4.2 Pull-based scraping — the Prometheus model

Now the second concept: how does the number get from inside the node to the monitoring system? There are two philosophies.

In a **push** model, the application actively sends its metrics out to a collector — "here are my numbers" — every few seconds. In a **pull** (or scrape) model, the application merely exposes its current numbers at a known location, and a central server periodically comes and reads them. The application is passive; it does not know or care who is watching.

Prometheus[4] is the de-facto standard monitoring system in this space, and it is built around the **pull** model. A Prometheus server is configured with a list of targets (addresses to scrape). On a fixed interval — say every 15 seconds — it visits each target, reads the current value of every metric, stamps it with the time, and stores the whole time-series in its own database. You then query and graph that database (commonly through Grafana, a dashboard tool).

We discuss why pull and not push in §42.4.5. First, Hathor's exact mechanism — which has a twist.

42.4.3 Two objects: `Metrics` and `PrometheusMetricsExporter`

`hathor-core` splits the job in two. One object collects the numbers from inside the node; a second object exports them in Prometheus format. Keeping them separate means the node always tracks its own metrics (cheap, always on) even when nobody has turned Prometheus export on.

The collector — `Metrics` (`hathor/metrics.py:58`). This is a `@dataclass` holding every number the node tracks, plus the logic to refresh them. Its fields are the metrics. A sampling of them (`metrics.py:67–118`):

```
@dataclass
class Metrics:
    pubsub: PubSubManager
    avg_time_between_blocks: int
    connections: ConnectionsManager
    tx_storage: TransactionStorage
    reactor: Reactor

    transactions: int = 0          # tx count in the network
    blocks: int = 0                # block count
    best_block_height: int = 0     # height of the best chain
    hash_rate: float = 0.0         # network hash rate
    peers: int = 0                 # peers connected
    best_block_weight: float = 0   # weight of the best block
    # ... websocket_connections, completed_jobs (stratum),
    #     rocksdb_cfs_sizes, transaction_cache_hits/misses,
    #     connected_peers, handshaking_peers, known_peers, ...
```

Notice it holds references to the live node components — the pub-sub bus, the connections manager, the transaction storage. That is how it gets fresh numbers: it asks them.

`Metrics` refreshes itself two ways, and the split between them is worth understanding because it is a recurring pattern for "how do I keep a number current."

Event-driven updates. In `subscribe()` (`metrics.py:166`) the metrics object registers with the pub-sub bus[3] for a handful of events — `NETWORK_NEW_TX_ACCEPTED`, the peer connect/disconnect events. When such an event fires, `handle_publish()` (`metrics.py:181`) updates the relevant numbers immediately. A new block accepted? Refresh the block count, hash rate, best-block weight and height on the spot (`metrics.py:189–194`). A peer connected? Update the peer counts. These numbers are kept exact because the node tells the metrics object the moment they change.

Polled updates. Some numbers are too expensive or too fiddly to recompute on every event, so they are sampled on a timer. In `__post_init__` (`metrics.py:120`) a `LoopingCall` is created bound to `_collect_data` (`metrics.py:130`), and `start()` launches it (`metrics.py:159`) at an interval taken from settings (`METRICS_COLLECT_DATA_INTERVAL`). Every tick, `_collect_data` (`metrics.py:300`) refreshes the WebSocket numbers, the Stratum mining numbers, the transaction-cache hit/miss counts, the per-peer connection metrics, and — less often, gated on block height — the RocksDB column-family sizes (`set_tx_storage_data`, `metrics.py:282`).

So: cheap-and-exact numbers update on the event that changes them; expensive numbers are polled. The result is one `Metrics` object whose fields are always reasonably current. Crucially, all of this runs whether or not Prometheus is enabled. The node tracks its own vital signs by default.

The exporter — `PrometheusMetricsExporter` (`hathor/prometheus.py:75`). This is the optional second half, built only when the operator passes `--prometheus`. Its job is to take the numbers out of the `Metrics` object and publish them in Prometheus format. In `_initial_setup` (`prometheus.py:124`) it creates a Prometheus `CollectorRegistry` and one `Gauge` object per metric (`prometheus.py:134–135`), driven by the `METRIC_INFO` dictionary at the top of the file (`prometheus.py:29`) that maps each metric name to a human description. It also sets up labelled gauges for the per-peer metrics, the RocksDB sizes, and Python garbage-collection stats (`prometheus.py:137–182`).

It too runs on a `LoopingCall` (`prometheus.py:117`, started at `prometheus.py:188`). Each tick, `set_new_metrics` (`prometheus.py:191`) copies every current value out of the `Metrics` object into the matching Prometheus `Gauge`:

```python
def set_new_metrics(self) -> None:
    for metric_name in METRIC_INFO.keys():
        self.metric_gauges[metric_name].set(getattr(self.metrics, metric_name))
    self._set_rocksdb_tx_storage_metrics()
    self._set_new_peer_connection_metrics()
    write_to_textfile(self.filepath, self.registry)   # prometheus.py:200
```

42.4.4 The twist: file-based export, not an HTTP endpoint

Here is where Hathor diverges from the textbook Prometheus setup, and it is worth stating plainly because the prompt's framing ("exposes a Prometheus endpoint") is not what the code does. Look at the last line above: `write_to_textfile`. The exporter does

not open an HTTP port that Prometheus scrapes directly. Instead, every interval it writes the current metrics to a file on disk — a `.prom` text file (`prometheus.py:79`, default filename `hathor.prom`), in a `prometheus/` directory under the node's data path (`resources_builder.py:104`).

How does Prometheus then read it? Through the **node_exporter** convention. `node_exporter` is a separate, standard Prometheus component (a small daemon you run alongside the node) with a textfile collector feature: point it at a directory of `.prom` files and it serves their contents over HTTP for Prometheus to scrape. The class docstring says exactly this — "sends hathor metrics to a node exporter that will be read by Prometheus" (`prometheus.py:76`). So the full path is:

```
Metrics object ──(LoopingCall)──▶ PrometheusMetricsExporter
      │  reads fields                    │ write_to_textfile()
      ▼                                  ▼
live node state                  hathor.prom   (file on disk)
                                       │
                                       ▼  (textfile collector)
                                 node_exporter ──HTTP──▶ Prometheus server
                                                         (scrapes every ~15s)
```

The end-to-end model is still pull — Prometheus scrapes `node_exporter` — but the node itself participates by writing a file, not by serving a port. This indirection keeps the node out of the business of running an HTTP metrics server and reuses a battle-tested external component for the scrape surface. (Be precise about this when you operate the node: enabling `--prometheus` alone does not give you a scrapeable URL; you must also run `node_exporter` pointed at the file.)

42.4.5 Why Prometheus, and why pull

The trade-off discussion the book promises for every technology choice.

Why metrics at all, versus just logging numbers? Covered in §42.2: a number that changes constantly is a time-series, not an event. Logging it floods the log and forces you to parse and re-aggregate; exposing it as a metric lets purpose-built tooling sample, store, and graph it.

Why pull, not push? The pull model has concrete operational advantages, which is why Prometheus chose it:

- **The monitoring system controls the load.** With pull, Prometheus decides how often to scrape; a misbehaving node cannot flood the collector by pushing too fast. Push systems need their own rate-limiting.
- **Targets are discoverable and their health is implicit.** If Prometheus tries to scrape a target and the scrape fails, that is a signal — "the node is down or unreachable." With push, a silent node is ambiguous: is it healthy-but-quiet, or dead?

- **Simplicity at the node.** The node only has to expose its current state; it does not need to know the monitoring server's address, manage retries, or buffer on network failure. (Hathor takes this even further by writing a plain file and letting `node_exporter` handle the HTTP.)

The cost is that pull does not fit every shape of system — short-lived batch jobs that finish before any scrape lands need a push gateway. A long-running daemon like a full node is the ideal pull target: it is always up and always has current numbers to read. Prometheus is the de-facto standard for exactly this kind of infrastructure monitoring, which is the practical reason to choose it: the surrounding ecosystem (Grafana dashboards, alerting rules, `node_exporter`) is mature and everyone already knows it.

42.5 The code, walked — the CPU profiler

42.5.1 The reactor-stall problem

Recall the cardinal rule from §42.3: the node runs on one reactor thread, and while any callback executes, the entire node is frozen. This makes a slow function uniquely dangerous here. In a multi-threaded server, one slow request ties up one thread; the others keep serving. In `hathor-core`, one slow callback ties up the only thread — every peer stalls, every API request hangs, every timer slips. A function that is merely "a bit slow" in isolation can, if it sits on the hot path, throttle the whole node's throughput.

So the operationally interesting question is not "is this function fast in a benchmark" but **"where is the reactor thread actually spending its time, right now, in production?"** A normal profiler (run the program under `cProfile`, read the report after it exits) does not help: you cannot stop a production node to profile it, and the interesting behavior only appears under real load. You need a profiler you can switch on inside a live node, watch, and switch off — with low enough overhead that turning it on does not itself stall the thing you are measuring.

That is what `hathor/profiler/` provides.

42.5.2 `SimpleCPUProfiler` — a sampling marker stack

The engine is `SimpleCPUProfiler` (`hathor/profiler/cpu.py:48`). It is deliberately not a full statistical profiler; it is a lightweight marker system. The idea: wrap sections of code in a `begin`/`end` pair, accumulate how much CPU time each named section consumed, and periodically compute each section's share of CPU.

Two methods do the measuring. `mark_begin(key)` (`cpu.py:123`) pushes a `(key, time.process_time())` pair onto a stack. `mark_end(key)` (`cpu.py:132`) pops it, computes the elapsed CPU time, and adds it to the running total for that key:

```
def mark_end(self, key: str) -> bool:
    if not self.enabled:
        return False
    # ... (stack-matching checks omitted)
    dt = time.process_time() - cur_time
    self.measures[tuple(x[0] for x in self.stack)].add_time(dt)
    self.stack.pop()
```

Two details matter. First, it uses `time.process_time()`, which measures CPU time consumed by this process, not wall-clock time — so time the node spent waiting (on the network, on disk) does not count against a section. That is exactly right for finding CPU hogs. Second, the key it records is the whole current stack (`tuple(x[0] for x in self.stack)`), so nested sections form a call path — you see not just "this function was slow" but "this function was slow when called from that one."

The accounting per section lives in `ProcItem` (`cpu.py:28`), which tracks `total_time` and, after each `update`, a `percent_cpu`. A `LoopingCall` runs `update()` (`cpu.py:156`) every few seconds (`update_interval=3.0` by default, `cpu.py:51`); it computes each section's CPU percentage over the interval, drops sections idle longer than `expiry` (15s), and sorts the list so the heaviest sections rise to the top (`cpu.py:181`). The output is `proc_list`: a CPU-sorted list of "what the reactor thread is spending its time on." Note the profiler is off by default (`enabled = False`, `cpu.py:88`); all the `mark_*` methods short-circuit immediately when disabled, so the overhead of leaving the hooks in the code is near zero.

42.5.3 Where the marks come from — the `@profiler` decorator and `SiteProfiler`

You do not litter the codebase with `mark_begin`/`mark_end` by hand. The profiler exposes a decorator[5], `profiler(key)` (`cpu.py:190`), that wraps any function so it is automatically marked on entry and exit:

```
@cpu.profiler('http-api')                                  # site.py:45
@cpu.profiler(key=lambda self, request: request.path.decode())
@cpu.profiler(key=lambda self, request: self._get_client_ip(request))
def getResourceFor(self, request):
    return super().getResourceFor(request)
```

This is `SiteProfiler` (`hathor/profiler/site.py:27`), a subclass of Twisted's `server.Site` (the HTTP server object). By overriding `getResourceFor` — the method Twisted calls to route every incoming HTTP request — and stacking three `@profiler` decorators on it, every API request is automatically timed three ways: under the global key `http-api`, under its URL path, and under the client IP. The result is that the profiler's output naturally groups CPU time by API endpoint and by caller — answering "which API is eating the reactor" without any manual instrumentation. `SiteProfiler` is the HTTP server the node actually installs when the status server is built (`resources_builder.py:343`).

42.5.4 The `/top` API and `hathor-cli top`

The profiler is controlled and read over a REST resource, `CPUProfilerResource` (`hathor/profiler/resources/cpu_profiler.py:31`), mounted at `/top`. A `POST` with body `start`, `stop`, or `reset` toggles the profiler (`cpu_profiler.py:43`); a `GET` returns the current `proc_list` as JSON — each section with its `percent_cpu` and `total_time` (`cpu_profiler.py:76`). So an operator can turn profiling on in a live node, sample for a while, and turn it off, all over HTTP.

Reading raw JSON is unpleasant, so there is a viewer: `hathor-cli top` (`hathor_cli/top.py`), registered as a subcommand "CPU profiler viewer" (`hathor_cli/main.py:76`). It is a **curses**[6] terminal UI — modelled on the Unix `top` command — that polls the node's `/top` endpoint every couple of seconds via an async `ProfileAPIClient` (`top.py:658`) and redraws a live, CPU-sorted table of where the reactor thread is spending its time, with keys to start/stop/reset the profiler remotely (`top.py:489`). It is, in effect, a `top` for the inside of the node: instead of OS processes, you see the node's own code sections ranked by CPU.

Tied together: the reactor is single-threaded, so a slow callback stalls everything (§42.5.1); the profiler measures CPU time per code section with negligible overhead when off (§42.5.2); the `@profiler` decorator and `SiteProfiler` instrument the HTTP path automatically (§42.5.3); and `hathor-cli top` lets an operator watch the ranking live and pinpoint the offending section (§42.5.4).

42.6 The code, walked — the health-check endpoint

42.6.1 Liveness versus readiness

The last question — is the node OK? — is the simplest to ask and the most consequential to answer, because the answer is usually consumed not by a human but by an **orchestrator**[7] like Kubernetes that will act on it automatically: restart the node, or stop sending it traffic.

Orchestrators distinguish two flavours of "OK," and the distinction is the heart of this section:

- **Liveness** — is the process alive at all? Is it stuck, deadlocked, crashed-but-not-exited? A failing liveness check tells the orchestrator: **kill and restart this container.** The right response to "not alive" is a restart, because a hung process will not fix itself.
- **Readiness** — is the process ready to do useful work? It may be perfectly alive yet not ready — for a node, the textbook case is still syncing: the process is healthy, but its view of the ledger is stale, so it should not be answering wallet queries with out-of-date data. A failing readiness check tells the orchestrator:

the node is fine, leave it running, but do not route traffic to it yet. The right response to "not ready" is to wait, not to restart — a restart would only make it start syncing from scratch again.

Confusing the two is a classic operational bug: wire a "still syncing" node into a liveness probe and the orchestrator will restart it forever, never letting it finish syncing. The whole point of separating the probes is that restart and withhold traffic are different remedies for different problems.

42.6.2 What "healthy" means for a Hathor node

The logic both endpoints share lives on the manager: `is_sync_healthy()` (`hathor/manager.py:923`). A node is considered healthy when **two** conditions both hold:

```
def is_sync_healthy(self) -> tuple[bool, Optional[str]]:
    if not self.has_recent_activity():
        return False, HathorManager.UnhealthinessReason.NO_RECENT_ACTIVITY
    if not self.connections.has_synced_peer():
        return False, HathorManager.UnhealthinessReason.NO_SYNCED_PEER
    return True, None
```

1. **Recent block activity.** `has_recent_activity()` (`manager.py:908`) checks that the newest block in storage has a timestamp within a tolerance of now — specifically `P2P_RECENT_ACTIVITY_THRESHOLD_MULTIPLIER × AVG_TIME_BETWEEN_BLOCKS`. If the node's latest block is too old, the node is not keeping up with the chain, so it is unhealthy.
2. **At least one synced peer.** `has_synced_peer()` (`manager.py:648` / `connections.has_synced_peer()`) checks the node has a peer it is actually in sync with. A node alone in the dark — connected to no one, or to no one current — cannot trust its own view of the ledger, so it is unhealthy.

Both must pass; the failure carries a human-readable reason from the `UnhealthinessReason` enum (`manager.py:91`): "Node doesn't have recent blocks" or "Node doesn't have a synced peer." This is a sync-health definition: a Hathor node is "healthy" precisely when it is keeping up with the network.

42.6.3 The two resources

`hathor-core` mounts two endpoints, both driven by that same `is_sync_healthy()`:

`/health` — `HealthcheckResource` (`hathor/healthcheck/resources/healthcheck.py:31`). This is the richer endpoint, built on the third-party **`healthcheck`** library (the `python-healthchecklib` dependency, `pyproject.toml:83`; the library is introduced where dependencies are catalogued, Chapter 13). The library models a service as a set of named components, each with its own check; here there is one component, `sync` (`healthcheck.py:70`), whose check calls `is_sync_healthy()` (`healthcheck.py:21`). The response is structured JSON listing each component, its pass/fail status, and the

reason — the format many monitoring tools expect. By default it returns HTTP **200** when healthy and **503** when not (`healthcheck.py:53`); a `strict_status_code=1` query parameter forces 200 always, for tools that key only off the body (`healthcheck.py:48`).

`/p2p/readiness` — `HealthcheckReadinessResource` (`hathor/p2p/resources/healthcheck.py:8`). A leaner endpoint with the same underlying check, returning a minimal `{success: true}` / `{success: false, reason: ...}` and the same 200/503 split (`p2p/resources/healthcheck.py:14`). The path and name make its intent explicit: this is the **readiness** probe — "is the node caught up enough to serve traffic?"

The shared `is_sync_healthy()` answers both the liveness-style `/health` and the readiness-style `/p2p/readiness`. In a Kubernetes deployment you would wire `/p2p/readiness` to the readiness probe (so traffic is withheld while the node syncs) and use `/health` for broader monitoring or a liveness probe — choosing thresholds to fit, as the `/p2p/readiness` OpenAPI note about `P2P_RECENT_ACTIVITY_THRESHOLD_MULTIPLIER` spells out (`p2p/resources/healthcheck.py:61`).

42.7 How it plugs into the lifecycle

Pulling the wiring together, in lifecycle order:

1. **Construction.** When the builder assembles the manager (Chapter 24), the manager constructs its `Metrics` object in `__init__` (`manager.py:212`), handing it the live pub-sub, connections, and storage references.
2. **Start.** In `HathorManager.start()` (Chapter 29), `self.metrics.start()` runs (`manager.py:325`): the metrics object subscribes to its pub-sub events and launches its `_collect_data` `LoopingCall`. From here the node is always tracking its own numbers, Prometheus on or off.
3. **Prometheus (optional).** If `--prometheus` was passed, the `ResourcesBuilder` (Chapter 24) builds a `PrometheusMetricsExporter` (`resources_builder.py:86–108`), pointing it at the `Metrics` object and a `prometheus/` directory under the data path, and starts its file-writing `LoopingCall`. An external `node_exporter` then exposes that file for a Prometheus server to scrape.
4. **REST resources mounted.** The `ResourcesBuilder` (Chapter 24) also mounts the observability HTTP resources into the API tree: `/profiler` and `/top` for the profiler (`resources_builder.py:215–216`), `/health` for the health check (`resources_builder.py:218`), and `/p2p/readiness` for readiness (`resources_builder.py:239`). The status server it installs is the `SiteProfiler` (`resources_builder.py:343`), so the HTTP path is profiled automatically.

5. **In production.** A Prometheus server scrapes the node's metrics on its own schedule; an operator runs `hathor-cli top` to watch CPU; an orchestrator probes `/p2p/readiness` and `/health` to decide whether to route traffic or restart. None of these touch the ledger; all of them keep the node observable while it runs untouched.

Recap

Surface	Question it answers	Shape	Key code	Consumed by
structured logs (Ch 17)	what happened?	event stream	`hathor/logging`	a human / log search
Metrics + Prometheus	how much, how fast?	counters / gauges	`metrics.py:58`, `prometheus.py:75`	Prometheus (via `node_exporter`)
CPU profiler	why is it slow?	CPU-time per code section	`cpu.py:48`, `site.py:27`, `/top`	`hathor-cli top` / a human
Health check	is it alive / ready?	pass/fail + reason	`manager.py:923`, `/health`, `/p2p/readiness`	an orchestrator (Kubernetes)

The node we built across Part II now has windows. Metrics expose its vital signs as numbers a monitoring system samples and graphs; the profiler lets an operator watch where the single reactor thread spends its CPU and find the callback that is starving everything else; the health-check endpoints give an orchestrator a one-bit answer — keep running, withhold traffic, or restart — built on whether the node is keeping up with the network. Each answers a question logs cannot, and together with logs (Chapter 17) they complete the operator's view of a running node.

One package of Part II remains. Everything so far has been about the production node and the surfaces for operating it. The final chapter turns inward, to the infrastructure that lets the project test all of this deterministically: **Chapter 43 — Determinism for tests**, where `hathor/simulator/` and `hathor/dag_builder/` spin up an entire in-memory network with a controllable clock, so a four-block reorg or a sync race can be reproduced on demand rather than hunted in the wild.

---

1. The reactor is the heart of the Twisted framework: a single event loop that waits for events (data on a socket, a timer firing) and calls the one piece of your code registered to handle each. The whole node runs on it, on one thread. Full treatment in Chapters 2 and 16. ↩
2. A `LoopingCall` is Twisted's periodic timer: you give it a function and an interval, and it calls that function every interval seconds on the reactor thread. Because it shares the reactor, the function must be quick — a slow one blocks the node. ↩
3. Pub-sub (publish–subscribe) is a messaging pattern: publishers announce events without knowing who listens, and subscribers register interest in event types. The metrics object subscribes to events like "new tx accepted" to update its numbers on the spot. Full treatment in Chapter 30. ↩
4. **Prometheus** is the de-facto-standard open-source monitoring system for infrastructure. It pulls (scrapes) numeric metrics from targets on a fixed interval, stores them as time-series, and lets you query and graph them (commonly via Grafana). Covered in §42.4. ↩
5. A decorator is a function that wraps another function to add behaviour without changing its body — here, to mark the start and end of a timed section. Full treatment in Chapter 4. ↩
6. curses is a standard library for building text-based, full-screen terminal user interfaces (menus, live-updating tables) instead of plain scrolling output. `hathor-cli top` uses it to draw a live `top`-style dashboard. ↩
7. An orchestrator is a system (Kubernetes is the common one) that runs and supervises containerized services automatically — starting them, restarting failed ones, and routing traffic only to healthy ones. It learns a service's state by calling its health-check endpoints. ↩